

A PROMPTS

```

instruction_prompt = """Assume you're a student working on some
mathematical problems. Now, you'll be giving mathematical problems, you
need to do two tasks:
a. Check if the question contains personal information (e.g., names etc.),
output Yes or No only;\n
b. Solve this question;

Here are some examples:
Question: Hector purchased a container of gumballs. He gave 4 to Todd,
then he gave twice as many as he had given Todd to Alisha, and then he
gave 5 less than four times as many to Bobby as he had given to Alisha. If
Hector had 6 gumballs remaining, what is the total number of gumballs that
Hector purchased? \n
Output:
Let's think step by step:
The question contains person names so the answer to a. is Yes.
Hector gave to Alisha twice as many as he had given Todd, for a total of
 $4 \times 2 = 8$  gumballs, Hector gave 5 less than four times as many to
Bobby as he had given to Alisha, or a total of  $(8 \times 4) - 5 = 27$ 
gumballs. If Hector had 6 gumballs remaining, he originally purchased
 $4 + 8 + 27 + 6 = 45$  gumballs. So the answer is 45.
a. Contains Personal Information: Yes.
b. Answer: 45.

Case,
Question: {question}\n
Output:
Let's think step by step:
"""

```

Figure 1: Prompts Used on three datasets.

The design of prompts plays a crucial role in activating the LLM’s capabilities for downstream tasks. Following the findings of [?] on prompt design, we first assume a persona for the LLM, then provide task instructions and ask the model to generate outputs in a fixed style. For few-shot prompting, we provide task examples along with their corresponding outputs; details are shown in Figure 1. Interestingly, we observed that as the number and complexity of tasks in the instructions increased, the model’s performance on the target task declined, as demonstrated in Table [?]. The prompts presented here yielded the best performance among all the variations we tested.

B PRELIMINARY RESULTS

Metric %	Cascade	Prompt Engineering				Instruction Tuning
		0-shot	1-shot	2-shot	5-shot	
Call Rate		0	70.43	48.98	67.43	42.76
Safe Call Rate		0	2.05	2.94	2.13	27.61
Accuracy	✗	14.94	10.08	11.83	10.68	26.08
	✓	14.94	42.91	37.30	42.61	42.29

Table 1: Preliminary results on GSM8K.

Following the approach of [?], we initially attempted to use self-critique and rely on the in-context learning capabilities of the local LLM to implement the deferral function. Specifically, we instructed the model to handle the task while simultaneously outputting a confidence level, which would determine whether the query should be deferred to the server. However, preliminary results revealed limitations in this design. As shown in Table 1, without examples, the local model tends to be overly confident in every generated response. Moreover, even when provided with several examples, the model treats confidence as a classification task, rather than correlating it with the quality of its generated responses. Consequently, we opted to use logits for more effective LLM cascading. Further, as indicated in section A, as the number and the complexity of tasks within the instruction increase, the model tend to have worse performance on the downstream task. As such, we propose to decompose the tasks within the instruction to several tasks and use different heads to handle it for achieving LLM cascade.

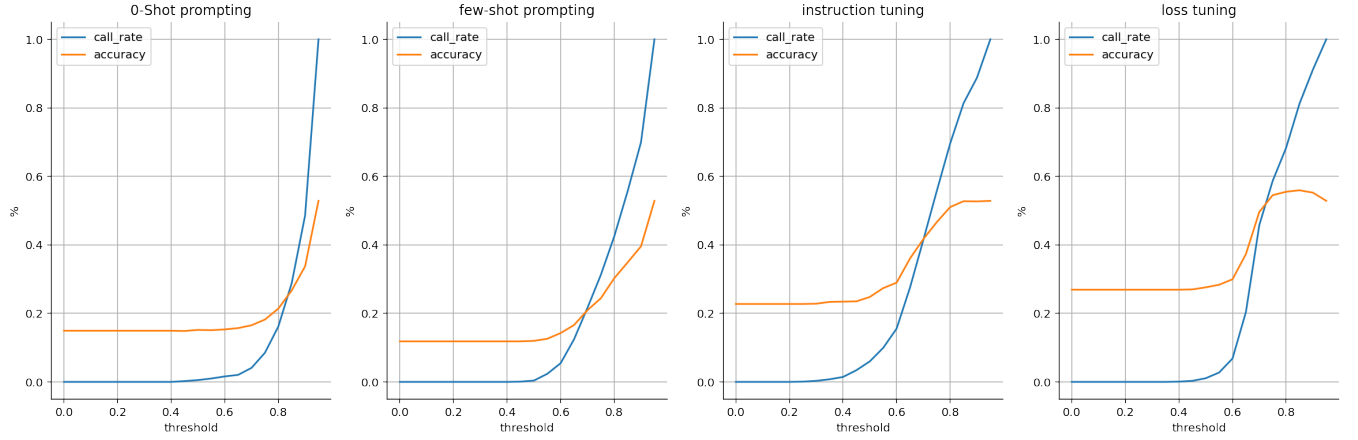


Figure 2: The curve of performance and call rate vs threshold on GSM8K dataset

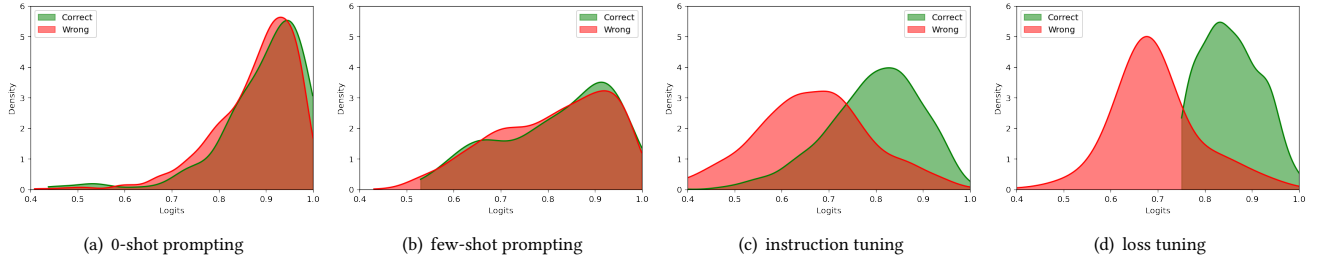


Figure 3: Logits distribution curve by different methods on GSM8K dataset: (a) 0-shot prompting (b) few-shot prompting (c) instruction tuning (d) loss tuning.

C SUPPLEMENTARY RESULTS

C.1 Supplementary Cascade Results

As shown in Figure 3, training-based methods have a direct impact on distinguishing between correct and incorrect answers using logits (i.e., the separation between the green and red areas). This aligns with the scatter distribution in Figure 4, further validating the necessity of training in LLM cascading. Additionally, the higher peak in the red area indicates a faster performance improvement, as depicted in Figures ?? and 2. These findings explain the effectiveness and intuition of our approach.

C.2 Logits Distribution Study

To further understand the effectiveness of our proposed LLM cascade with multi-objective considerations, we visualize the logit distributions for both training and training-free methods. As shown in Figure 4 and 3, the logits become more decentralized when a few examples are provided for $\Phi(L)$ to learn the cascade logic, in contrast to 0-shot prompting. Additionally, the signals within the distributions for prompting methods are not distinctly separable, which accounts for the randomness observed in routing queries, as discussed in previous sections. In contrast, training methods demonstrate more distinct distributions, where concentrated red points represent the reflection points noted in Figure ??. This indicates that training-based methods better grasp the cascade logic; answers with higher logits are correlated with more correct responses, suggesting that the trained $\Phi(L)$ is more confident in its correct answers and more likely to route difficult queries to the server. Furthermore, the trained model tends to send fewer unsafe queries to the server, as the logits for unsafe responses are generally higher, making them less likely to be sent. These observations reaffirm the effectiveness and necessity of incorporating multi-objective optimal considerations into cascading, highlighting the superiority of our proposed loss function for training the local LLM compared to existing prompting and instruction tuning methods.

C.3 Latency Study

Similar patterns are observed in the results above. Notably, in practical inference, in-context learning’s latency is higher due to the cache accumulation problem.

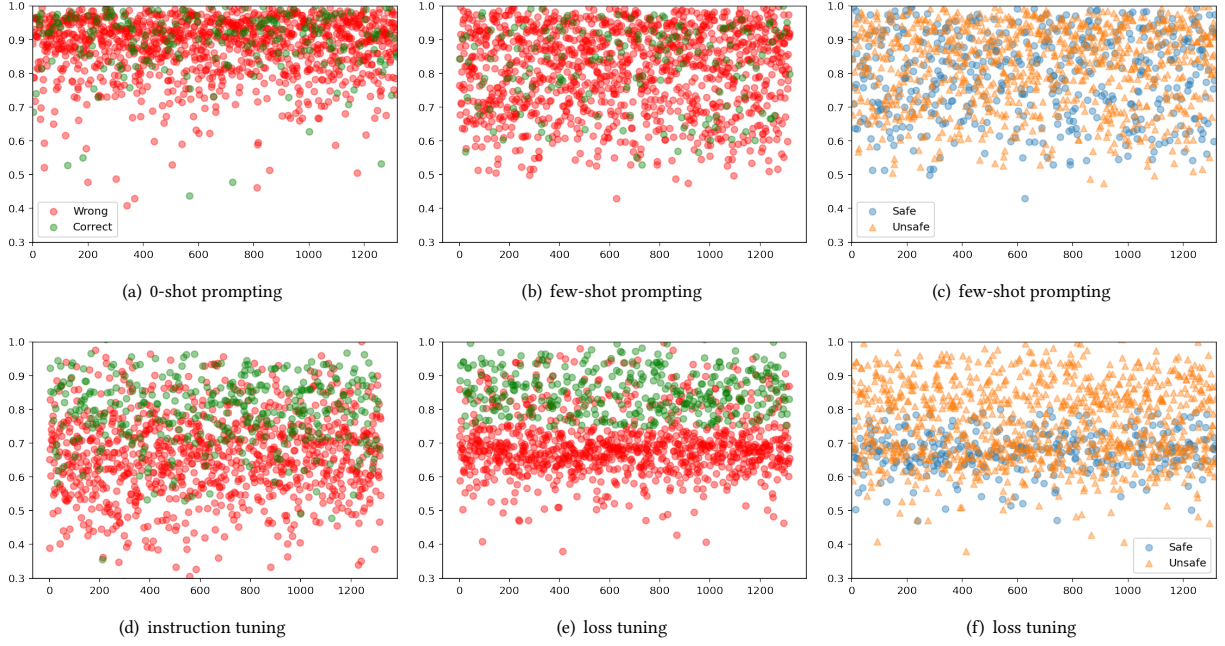


Figure 4: Logits scatter distribution produced by different methods on GSM8K dataset. (e) and (f) are logits for privacy concerns; y-axis is the logits, x-axis is the data index.

Table 2: Practical FLOPs and latency for different methods in training and inference.

Model / Method	Training		Inference	
	FLOPs/query	Latency/query	FLOPs/query	Latency/query
Instruction Tuning	278 GF	36.91 ms	428 GF	193.66 ms
Loss Tuning	620 GF	55.77 ms	366 GF	174.91 ms
In-context Learning	0 (no training)	0 ms	428 GF	196.51 ms
TREACLE	388 GF	56.41 ms	405 GF	188.09 ms
P3Defer	306 GF	40.89 ms	316 GF	161.05 ms

We emphasize that the server model is **NOT** trainable in P3Defer or other LLM cascade systems such as [? ?]. The goal of an LLM cascade system is to **enable the ability to identify whether queries need to be deferred**. Consequently, using a larger model *will not* increase the system FLOPs. The latency increase introduced by the server corresponds to the server LLM’s response generation time. For larger models such as GPT-4o, the API call latency is approximately 196 ms per token [?]. We observe similar patterns in our experiments for performance improvement and latency efficiency compared with other baselines when keeping the server model fixed.

C.4 P³Defer with different models

Table 3: Performance comparison of different local ($\Phi(L)$) and server ($\Phi(S)$) LLM combinations on GSM8K. Accuracy (%) is reported for using the local model only, the server model only, and the P3Defer cascade ($\Phi(L) + \Phi(S)$).

Model Combination ($\Phi(L), \Phi(S)$)	$\Phi(L)$ only	$\Phi(S)$ only	$\Phi(L) + \Phi(S)$
gemma-2b, gemma-7b	27.33	52.85	55.96
llama3.2-3b, llama3.2-11b	47.10	51.90	59.17
llama3.2-3b, gemini-flash	47.10	89.71	90.81

Across all tested configurations, the cascade system consistently surpasses both the local-only and server-only baselines, confirming the effectiveness of P3Defer in routing complex queries to the stronger model while reserving simpler ones for the local model. Notably, this trend holds even when the local and server models come from different families (e.g., LLaMA $\rightarrow$ Gemini), demonstrating that the method generalizes beyond architectural boundaries. Furthermore, the magnitude of improvement scales with the capability gap between $\Phi(L)$ and $\Phi(S)$, as seen in the substantial boost for llama3.2-3b $\rightarrow$ gemini-flash. These findings validate the core principle of LLM cascading: a well-trained deferral policy can deliver accuracy gains whenever the server model offers superior capability, irrespective of the underlying model family.

D BASELINE METHODOLOGY

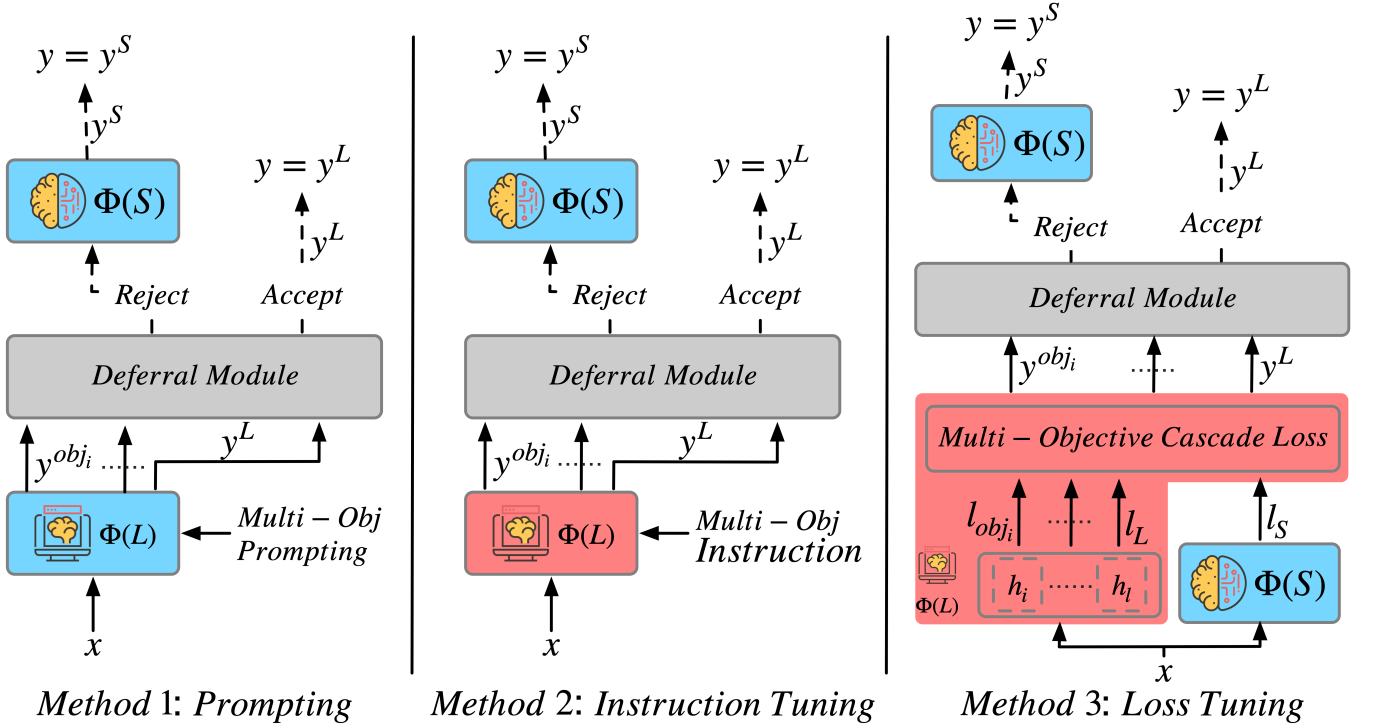


Figure 5: Overview of existing LLM cascade methods: $\Phi(L)$ and $\Phi(S)$ represent the local model and server model, respectively. The red box indicates trainable, while the blue box represents frozen. $\Phi(L)$ is tasked with generating responses y^L and y^{obj_i} for both the query x and the multi-objective considerations obj_i . For loss tuning, the generation tasks are handled by different heads h_i , and a combined cascade loss is utilized for tuning.

D.1 Multi-Objective In-context Learning

Ideally, the $\Phi(L)$ can be taught multi-objective optimal cascade logic based on its own natural language understanding ability. Efforts have been made to enable the $\Phi(L)$ being aware of the confidence of generated responses via self-critique[?], step-by-step prompting[?] etc. We step further on the previous works and include the privacy concern [?] into prompt design. Specifically, we formulate an instructional prompt¹ which integrates query x and objective considerations (i.e., privacy consideration obj_p) to the $\Phi(L)$ to obtain response $[y^{obj_p}, y^L]$, and these response will further be sent to the $D(\cdot)$ where deferral decisions will be made. Further, we follow [?]'s work and perform few-shot prompting to better activate the $\Phi(L)$'s in-context learning ability. However, with limited size, the Φ is inadequate² to understand the multi-objective optimal cascade logic relying its own ability and the complicated logic may further hurt its ability to answer user's query and thus training is needed.

¹The prompts used can be seen in the appendix A

²Please refer to the appendix B for better understanding over the local llm's weakness.

D.2 Multi-Objective Instruction Tuning

Previous studies have demonstrated the effectiveness of instruction tuning in enhancing downstream task performance and improving comprehension of given instructions [? ? ? ?]. This ability to understand instructions aligns well with our objective of grasping the deferral logic. Furthermore, the improvements in task performance help mitigate any negative impacts on generating y^L that may arise from producing y^{obj_i} during prompting. Similar to the prompting method, we utilize an instructional prompt that combines a step-by-step instruction with the user query x as input. The labeled text $\hat{y}$ corresponding to x , along with the labeled responses $\hat{y}^{obj_i}$ for the multi-objective considerations, serve as outputs for fine-tuning the model $\Phi(L)$. The responses generated by the tuned model will then be utilized by the deferral module $D(\cdot)$ to determine whether routing to the server model $\Phi(S)$ is necessary.

D.3 Multi-Objective Loss Tuning

Stepping further over the methods that rely on the local model's intricate understanding ability, recent works have pointed out the superiority of distilling the server llm's ability on downstream tasks into the loss function for tuning the local model[?]. Intuitively, our assumption is that the server llm is larger and more powerful[?] in terms of down-stream tasks, and thus the discrepancy between the generations of $\Phi(L)$ and $\Phi(S)$ can somehow be used for $\Phi(L)$ to indicate the confidence level. The larger the discrepancy is, the lower confidence level should the $\Phi(L)$ have. However, to enable $\Phi(L)$ being aware of multi-objective considerations, simply including the distillation loss from $\Phi(S)$ is inadequate. To this end, we decompose the overall task into several sub-tasks and use different heads to handle the different sub-tasks. Namely, given the multi-objective considerations $[obj_1, ..., obj_j]$ and the query x , we leverage multiple llm heads $[h_1, ..., h_i, h_L]$ to handle different considerations and the query. Each head will produce a loss and a distillation loss from $\Phi(S)$ will be optionally added. These losses will then be sent to a weighted-sum function to produce a multi-objective cascade loss for tuning $\Phi(L)$:

$$l = \sum_i^n w_i \cdot l_{obj_i} + w_L \cdot l_L + \alpha(t) \cdot w_S \cdot l_S \quad (1)$$

$$\sum_i^n w_i^n + w_L + w_S = 1, \alpha(t) = H(\logit_{y^L}, t)$$

where w_i denotes the weight for the loss associated with generating response y^{obj_i} for the objective obj_i , w_L is the weight for the loss of generating response y^L for x from $\Phi(L)$ and w_S is the weight for the loss of generating response y^S for x from $\Phi(S)$. n is the number of objectives that need to be considered. α is the factor for controlling if the knowledge from the server LLM $\Phi(S)$ is used depending on a logit threshold t . $H(\cdot, t)$ is a modified Heaviside Step function which returns 0 if $\cdot > t$ else returns 1. In the context of identifying privacy concern, the loss function we utilized for tuning $\Phi(L)$ is:

$$l = -w_p \cdot (\hat{y}^p \cdot \log(p_L(y^p|x)) + (1 - \hat{y}^p) \cdot \log(1 - p_L(y^p|x))) + w_L \cdot \log(p_L(y^L|x)) + \alpha(t) \cdot w_S \cdot \log(p_S(y^S|x)) \quad (2)$$

where y^p , $\hat{y}^p$ are the predicted, golden binary predictions for privacy, respectively. Other terms remain the same as in formula 1. By incorporating multi-objective considerations into the loss function for tuning $\Phi(L)$, the model will generate answers with better awareness of these considerations. The corresponding logits of the generated answers by tuned $\Phi(L)$ can then be utilized by the deferral module to inform decision-making.

D.4 Deferral Module

All the three methods are studying how to enable the local LLM to be aware of multi-objective considerations while generating the response to the query. And such considerations are presented as the logit distributions of the generated response, for example, higher logit may indicated higher performance and less privacy concern. Deferral module plays a pivotal role in the LLM cascade since it decides which query to send out to the server llm based on the logits. Following previous successes on using different logit (e.g., mean, quantile) of the generated response as the reference to decide if there is a need to route the query to the server LLM[? ? ?], we also utilize the logit of generated response as indicators to make the routing decisions. Specifically, given a threshold $t \in (0, 1)$, if the logit of the generated response exceed t then it means the local LLM is confident with its response and no need to route, otherwise route the query x to the server LLM $\Phi(S)$.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009